

Python Cheat Sheet

Sentimentanalyse Grimm — Alle Befehle auf einen Blick

Kapitel 01 — Variablen und Datentypen

Was	Code	Beispiel / Ergebnis
Variable zuweisen	<code>name = wert</code>	<code>maerchen = 86</code>
Wert ausgeben	<code>print(variable)</code>	<code>print(maerchen)</code>
Mehrere Werte ausgeben	<code>print(a, b, c)</code>	<code>print('Es gibt', maerchen, 'Märchen.')</code>
Typ prüfen	<code>type(variable)</code>	<code>type(maerchen) → int</code>
Ganzzahl (Integer)	<code>int</code>	42
Kommazahl (Float)	<code>float</code>	3.14
Zeichenkette (String)	<code>str</code>	'Grimm'
Addition / Subtraktion	<code>+</code> / <code>-</code>	<code>3 + 5 → 8</code>
Multiplikation / Division	<code>*</code> / <code>/</code>	<code>7 / 2 → 3.5</code>
Kommentar	<code># Text</code>	<code># wird von Python ignoriert</code>

Strings werden mit Anführungszeichen geschrieben. Bei Floats wird ein Punkt statt Komma genutzt: 10.4 statt 10,4.

Kapitel 02 — Strings und Booleans

Strings (Zeichenketten)

Was	Code	Beispiel / Ergebnis
String erstellen	<code>"Text" oder 'Text'</code>	<code>titel = "Froschkönig"</code>
Länge	<code>len(text)</code>	<code>len("Grimm") → 5</code>
Einzelnes Zeichen (Index)	<code>text[position]</code>	<code>"Froschkönig"[0] → "F"</code>
Von hinten (neg. Index)	<code>text[-position]</code>	<code>"Hallo"[-1] → "o"</code>
Ausschnitt (Slice)	<code>text[start:ende]</code>	<code>"Froschkönig"[0:3] → "Fro"</code>
Ab Position	<code>text[start:]</code>	<code>"Hallo"[2:] → "llo"</code>
Bis Position	<code>text[:ende]</code>	<code>"Hallo"[:3] → "Hal"</code>
Strings verbinden	<code>text1 + text2</code>	<code>"Gebrüder " + "Grimm"</code>

Der Index beginnt bei 0. Beim Slice ist die Endposition nicht eingeschlossen.

Booleans (Wahrheitswerte)

Was	Code	Beispiel / Ergebnis
Wahr / Falsch	<code>True / False</code>	<code>gefunden = True</code>
Gleich / Ungleich	<code>==</code> / <code>!=</code>	<code>3 == 3 → True</code>
Kleiner / Größer	<code><</code> / <code>></code> / <code><=</code> / <code>>=</code>	<code>5 > 3 → True</code>
Enthalten in	<code>wert in text</code>	<code>"a" in "Hallo" → True</code>
Nicht enthalten	<code>wert not in text</code>	<code>"x" not in "Hallo" → True</code>

Kapitel 03 — Listen

Was	Code	Beispiel / Ergebnis
Liste erstellen	<code>[wert1, wert2, ...]</code>	<code>tiere = ["Wolf", "Fuchs"]</code>
Leere Liste	<code>[]</code>	<code>ergebnisse = []</code>
Element abrufen	<code>liste[position]</code>	<code>tiere[0] → "Wolf"</code>
Ausschnitt (Slice)	<code>liste[start:ende]</code>	<code>tiere[0:2]</code>
Element anhängen	<code>liste.append(wert)</code>	<code>tiere.append("Hase")</code>
Element entfernen	<code>del liste[position]</code>	<code>del tiere[0]</code>
Wert ersetzen	<code>liste[pos] = neuer_wert</code>	<code>tiere[0] = "Bär"</code>
Länge	<code>len(liste)</code>	<code>len(tiere) → 2</code>
Enthalten?	<code>wert in liste</code>	<code>"Wolf" in tiere → True</code>
Verschachtelte Liste	<code>liste[zeile][spalte]</code>	<code>regal[0][2]</code>

Listen sind veränderbar — Strings dagegen nicht. Methoden wie `.append()` werden mit Punkt an das Objekt gehängt.

Kapitel 04 — Texte einlesen und bereinigen

Datei einlesen

```
with open("maerchen.txt", "r", encoding="utf-8") as datei:
    text = datei.read()
```

Textbereinigung

Was	Code	Beispiel / Ergebnis
Kleinschreibung	<code>text.lower()</code>	<code>"Hallo".lower() → "hallo"</code>
Zeichen ersetzen	<code>text.replace(alt, neu)</code>	<code>"Hallo".replace(".", "")</code>
Mehrfach ersetzen	<code>.replace().replace()</code>	<code>text.replace(".", "").replace(",", "")</code>
In Wörter zerlegen	<code>text.split()</code>	<code>"Hallo Welt".split() → ["Hallo", "Welt"]</code>
Leerzeichen entfernen	<code>text.strip()</code>	<code>" Hallo ".strip() → "Hallo"</code>

Bibliotheken und Wörter zählen

Was	Code	Beispiel / Ergebnis
Bibliothek importieren	<code>import bibliothek</code>	<code>import string</code>
Mit Abkürzung	<code>import ... as ...</code>	<code>import pandas as pd</code>
Einzelnes importieren	<code>from ... import ...</code>	<code>from collections import Counter</code>
Wörter zählen	<code>Counter(liste)</code>	<code>haeufigkeiten = Counter(wort_liste)</code>
Häufigste Wörter	<code>.most_common(n)</code>	<code>haeufigkeiten.most_common(10)</code>

Kapitel 05 — Schleifen und Bedingungen

for-Schleife

```
for figur in figuren:
    print(figur)
```

Bedingungen (if / elif / else)

```
if anzahl > 200:
    print("Riesige Sammlung!")
elif anzahl > 50:
    print("Große Sammlung!")
else:
    print("Kleine Sammlung.")
```

Nützliche Prüfungen in Schleifen

Was	Code	Beispiel / Ergebnis
Ist es eine Zahl?	<code>text.isdigit()</code>	<code>"153".isdigit()</code> → True
Element entfernen	<code>liste.remove(wert)</code>	<code>wort_liste.remove("153")</code>
Nicht enthalten?	<code>wert not in liste</code>	<code>wort not in stopwords</code>

Häufiges Muster: Liste filtern

```
bereinigte_liste = []
for wort in wort_liste:
    if wort not in stopwords:
        bereinigte_liste.append(wort)
```

Kapitel 07 — Wörterbücher (Dictionaries) und Sentimentanalyse

Was	Code	Beispiel / Ergebnis
Wörterbuch erstellen	<code>{schluessel: wert, ...}</code>	<code>{"schön": 1, "böse": -1}</code>
Wert abrufen	<code>woerterbuch[schluessel]</code>	<code>lexikon["schön"]</code> → 1
Schlüssel prüfen	<code>schluessel in wb</code>	<code>"schön" in lexikon</code> → True
Wert mit Fallback	<code>wb.get(schluessel, standard)</code>	<code>lexikon.get("wald", 0)</code> → 0
Alle Schlüssel	<code>woerterbuch.keys()</code>	<code>lexikon.keys()</code>
Alle Werte	<code>woerterbuch.values()</code>	<code>lexikon.values()</code>
Schlüssel + Werte	<code>woerterbuch.items()</code>	<code>lexikon.items()</code>

Häufiges Muster: Sentiment Score berechnen

```
gesamt_score = 0
gefundene_woerter = []

for wort in gefilterte_liste:
    if wort in lexikon:
        wert = lexikon[word]
        gesamt_score = gesamt_score + wert
        gefundene_woerter.append(wort)
```

Kapitel 09 — Visualisierung mit matplotlib

Bibliothek importieren

```
import matplotlib.pyplot as plt
```

Was	Code	Beispiel / Ergebnis
Neue Grafik	<code>plt.figure(figsize=(b, h))</code>	<code>plt.figure(figsize=(10, 4))</code>
Liniendiagramm	<code>plt.plot(daten)</code>	<code>plt.plot(verlauf, color="steelblue")</code>
Balkendiagramm	<code>plt.bar(x, y)</code>	<code>plt.bar(namen, scores, color="green")</code>
Titel	<code>plt.title("...")</code>	<code>plt.title("Stimmungsverlauf")</code>
Achsenbeschriftung	<code>plt.xlabel / plt.ylabel</code>	<code>plt.xlabel("Abschnitte")</code>
Horizontale Linie	<code>plt.axhline(y)</code>	<code>plt.axhline(0, color="red", linestyle="--")</code>
Fläche füllen	<code>plt.fill_between(...)</code>	<code>plt.fill_between(range(...), verlauf, 0)</code>
Legende	<code>plt.legend()</code>	
Gitternetz	<code>plt.grid(True)</code>	<code>plt.grid(True, alpha=0.3)</code>
Layout anpassen	<code>plt.tight_layout()</code>	
Grafik anzeigen	<code>plt.show()</code>	

Mehrere Grafiken nebeneinander

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].plot(daten)
axes[0].set_title("Titel")
```

Häufige Muster — Zusammenfassung

Zähler hochzählen

```
zaehler = 0
for wort in liste:
    zaehler = zaehler + 1
```

Werte ansammeln

```
ergebnis = 0
for wert in werte_liste:
    ergebnis = ergebnis + wert
```

Stoppwörter und Zahlen entfernen

```
gefilterte_liste = []
for wort in wort_liste:
    if wort not in GERMAN_STOPWORDS and not wort.isdigit():
        gefilterte_liste.append(wort)
```

Stimmungsverlauf berechnen (Kap. 09)

```
verlauf = []
score = 0
zaehler = 0

for wort in wort_liste:
    if wort in GRIMM_SENTIMENT_LEXIKON:
        score = score + GRIMM_SENTIMENT_LEXIKON[word]
    zaehler = zaehler + 1
    if zaehler == 50:
        verlauf.append(score)
        score = 0
        zaehler = 0
```